

Lezione 3 - Pipeline, branch prediction e branchless programming

PIPELINE

◆ Latenza e throughput

Per analizzare le prestazioni di un programma introduciamo due concetti fondamentali:

- **latenza (latency)** → tempo che intercorre tra l'inizio e il completamento di una operazione
- **throughput** → numero di operazioni completate per unità di tempo

La latenza si misura spesso in **cicli di clock**, cioè quanti cicli servono per completare una istruzione.

Il throughput invece si misura in **IPC (instructions per cycle)**, cioè quante istruzioni vengono completate a ogni ciclo di clock.

◆ Esecuzione di una istruzione

Una istruzione non viene eseguita tutta insieme, ma attraversa diverse fasi:

- **fetch** → caricamento dell'istruzione dalla memoria
- **decode** → interpretazione dell'istruzione
- **execute** → esecuzione dell'operazione
- **memory** → eventuale accesso alla memoria
- **write** → scrittura del risultato

👉 queste fasi sono gestite da componenti diverse della CPU

◆ Esecuzione senza pipeline

Nel modello più semplice, ogni istruzione viene completata prima di iniziare la successiva.

👉 quindi:

- una istruzione occupa tutte le fasi
- la successiva può iniziare solo dopo

Questo porta a:

- latenza $\approx$ numero di fasi (es. 5 cicli)
- throughput basso (es. IPC $\approx$ 0.2)

◆ Pipeline

Osserviamo che le diverse fasi utilizzano componenti diverse della CPU.

👉 quindi è possibile iniziare una nuova istruzione prima che la precedente sia terminata.

Questo introduce la **pipeline**.

Con la pipeline, più istruzioni vengono eseguite contemporaneamente, ma in fasi diverse.

👉 esempio:

- istruzione 1 $\rightarrow$ execute
- istruzione 2 $\rightarrow$ decode
- istruzione 3 $\rightarrow$ fetch

◆ Effetto della pipeline

Per capire davvero la pipeline, immaginiamo una sequenza di istruzioni che attraversano le varie fasi:

ciclo	istruzione 1	istruzione 2	istruzione 3
1	fetch		
2	decode	fetch	
3	execute	decode	fetch
4	memory	execute	decode
5	write	memory	execute

ciclo	istruzione 1	istruzione 2	istruzione 3
6		write	memory
7			write

👉 ogni riga rappresenta un ciclo di clock

👉 ogni colonna una istruzione

Da questa rappresentazione si osserva che:

- più istruzioni sono in esecuzione contemporaneamente
- ogni istruzione si trova in una fase diversa
- le unità della CPU lavorano in parallelo

Dopo i primi cicli (*fase di riempimento*), la pipeline entra a regime:

👉 viene completata circa **una istruzione per ciclo**

👉 idealmente, in assenza di problemi (hazards)

◆ Osservazioni importanti

- la pipeline **non riduce la latenza** della singola istruzione
- la pipeline **aumenta il throughput**
- l'efficienza dipende dal fatto che le fasi siano indipendenti

📌 ARCHITETTURE AVANZATE

◆ Architetture superscalari

La pipeline permette di eseguire più istruzioni contemporaneamente, ma ogni ciclo completa al massimo **una istruzione**.

Le architetture **superscalari** estendono questo concetto:

👉 il processore può eseguire **più istruzioni nello stesso ciclo di clock**

Questo è possibile perché la CPU dispone di:

- più unità funzionali (ALU, FPU, ecc.)
- più pipeline parallele

In questo modo:

- più istruzioni indipendenti possono essere eseguite contemporaneamente
- il throughput aumenta

👉 quindi IPC può essere **maggiore di 1**

◆ Condizione importante

Per sfruttare il superscalare, le istruzioni devono essere:

👉 **indipendenti tra loro**

Se una istruzione dipende dal risultato della precedente, non può essere eseguita in parallelo.

◆ Out-of-order execution

Un ulteriore miglioramento è l'esecuzione **out-of-order**.

Normalmente le istruzioni vengono eseguite nell'ordine in cui compaiono nel codice.

Con l'out-of-order:

- 👉 il processore può cambiare l'ordine di esecuzione
- 👉 purché il risultato finale sia corretto

◆ Perché serve

Serve per evitare che la CPU resti ferma quando una istruzione è bloccata (ad esempio in attesa della memoria).

👉 esempio:

- istruzione 1 → deve aspettare un dato dalla memoria
- istruzione 2 → indipendente

👉 senza out-of-order → la CPU aspetta

👉 con out-of-order → esegue subito la 2

◆ Effetto

- migliore utilizzo delle risorse della CPU
- aumento del throughput
- riduzione dei tempi morti

◆ Idea generale

Possiamo vedere l'evoluzione così:

- pipeline → parallelismo tra fasi
- superscalare → parallelismo tra istruzioni
- out-of-order → riordino intelligente delle istruzioni

✳ tutti servono ad aumentare le prestazioni del processore

📌 HAZARDS (problemi della pipeline)

La pipeline migliora le prestazioni, ma introduce alcuni problemi chiamati **hazards**.

👉 gli hazards sono situazioni in cui la pipeline non può procedere normalmente.

◆ Structural hazards

I **structural hazards** si verificano quando più istruzioni vogliono usare **la stessa risorsa hardware** nello stesso momento.

👉 esempio:

- due istruzioni vogliono accedere alla memoria
- oppure vogliono usare la stessa unità funzionale (ALU, FPU, ecc.)

👉 problema:

- la CPU non può eseguire entrambe contemporaneamente
- una delle due deve aspettare

👉 conseguenza:

- si introducono ritardi (stall)
- si riduce il parallelismo

◆ Soluzione (idea)

Per ridurre i structural hazards si può:

- duplicare le risorse (più unità funzionali)
- migliorare l'organizzazione dell'hardware

👉 questo è uno dei motivi per cui esistono architetture superscalari

◆ Data hazards

I **data hazards** si verificano quando una istruzione dipende dal risultato di una precedente.



Esempio:

a = b + c;

d = a + e;

La seconda istruzione usa a , che viene calcolato dalla prima.

👉 problema:

- la seconda istruzione non può essere eseguita subito
- deve aspettare che la prima completi

👉 conseguenza:

- la pipeline si ferma (stall)
- si perde parallelismo

◆ Soluzioni (idea)

Per ridurre i data hazards si possono usare:

- **forwarding** → passare direttamente il risultato tra fasi della pipeline
- **riordinamento istruzioni** → eseguire prima istruzioni indipendenti

◆ Control hazards

I **control hazards** sono legati alle istruzioni di salto (if, branch).

 **Esempio:**

```
if (x > 0)
{
    a = b + c;
}
```

👉 problema:

- non si sa subito quale istruzione eseguire dopo

- la pipeline potrebbe caricare istruzioni sbagliate

👉 conseguenza:

- bisogna aspettare il risultato del branch
- oppure scartare istruzioni già caricate

◆ Effetto

I control hazards possono:

- rallentare la pipeline
- ridurre il throughput
- introdurre cicli inutili

◆ Idea generale

Gli hazards mostrano un limite della pipeline:

👉 non tutte le istruzioni possono essere eseguite in parallelo

👉 servono quindi tecniche per gestire questi problemi

📌 BRANCH PREDICTION

◆ Problema dei branch

Le istruzioni di salto (branch), come `if`, creano problemi nella pipeline.

💻 Esempio:

```
if (x > 0)
{
    a = b + c;
}
```


👉 problema:

- la CPU non sa subito quale istruzione eseguire dopo
- deve aspettare il risultato della condizione

👉 conseguenza:

- la pipeline si ferma (stall)
- oppure vengono caricate istruzioni sbagliate

◆ Idea della branch prediction

Per evitare di fermarsi, la CPU fa una **previsione (prediction)**:

👉 prova a indovinare quale sarà il risultato del branch

👉 quindi:

- continua a eseguire istruzioni
- senza aspettare il risultato reale

◆ Due casi

✓ previsione corretta

- la pipeline continua normalmente
- nessuna perdita di prestazioni

✗ previsione sbagliata

- le istruzioni eseguite vengono scartate
- si deve tornare indietro

👉 questo introduce un costo (*penalty*)

◆ Effetto sulle performance

- se la previsione è spesso corretta → grande aumento di performance
- se è spesso sbagliata → rallentamento

👉 quindi l'efficacia dipende dalla qualità della previsione

◆ Idea generale

La branch prediction serve per:

- ridurre i control hazards
- evitare stall della pipeline
- mantenere alto il throughput

✳ è una tecnica fondamentale nei processori moderni

📌 BRANCHLESS PROGRAMMING

◆ Idea

Le istruzioni di salto (branch) possono rallentare la pipeline a causa dei **control hazards**.

👉 per evitarli, possiamo scrivere codice **senza branch** (*branchless*).

💻 Esempio con branch

```
if (x > y)
    max = x;
else
    max = y;
```

👉 questo introduce un branch → possibile rallentamento



Versione branchless

```
max = (x > y) ? x : y;
```

👉 non c'è salto esplicito

👉 il codice è più lineare

◆ Idea di base

Il branchless programming consiste nel:

- eliminare gli `if` quando possibile
- usare espressioni che non interrompono il flusso della pipeline

◆ Perché funziona

- evita i control hazards
- evita errori di branch prediction
- mantiene il flusso continuo

◆ Attenzione

Il branchless programming:

- ✓ può migliorare le prestazioni
- ✗ non sempre è più leggibile
- ✗ non sempre è più veloce (dipende dal caso)

◆ Quando usarlo

- codice critico per le prestazioni

- cicli molto frequenti
- situazioni con molti branch

◆ Idea finale

Possiamo riassumere così:

- hazards → problemi della pipeline
- branch prediction → prova a ridurli
- branchless programming → li evita direttamente



Esempio pratico

Consideriamo un ciclo con un branch:

```
for (int i = 0; i < n; i++)  
{  
    if (v[i] > 0)  
        sum += v[i];  
}
```

👉 questo introduce un branch ad ogni iterazione



Versione branchless

```
for (int i = 0; i < n; i++)  
{  
    sum += (v[i] > 0) * v[i];  
}
```

👉 l'espressione `(v[i] > 0)` vale:

- 1 se vero
- 0 se falso

👉 quindi:

- se $v[i] > 0 \rightarrow$ somma $v[i]$
- altrimenti $\rightarrow$ somma 0

◆ Osservazione

- la versione con `if` può causare branch misprediction
- la versione branchless evita completamente il branch

👉 nei cicli molto grandi questo può migliorare le prestazioni